semester management system

RBAC

Item	Details
Team Name	Binary Brains
Scenario	Semester-Management-System-Database
Module	ICT 1407 Database Management System
Academic Year	First Year, 2023/2024
Submission Date	25th October 2026

No.	Member Name	Index Number	Additional Number
1	J.P.A.D.JAYAWEERA	1CT/2024/060	6672
2	C.D. WIJAYARATHNA	1CT/2024/135	6735
3	J M ACHINI IMALSHA	1CT/2024/055	6668
4	J.T.IlANGASINGHA	1CT/2024/054	6667
5	B.S MUSAFER	1CT/2024/087	6692

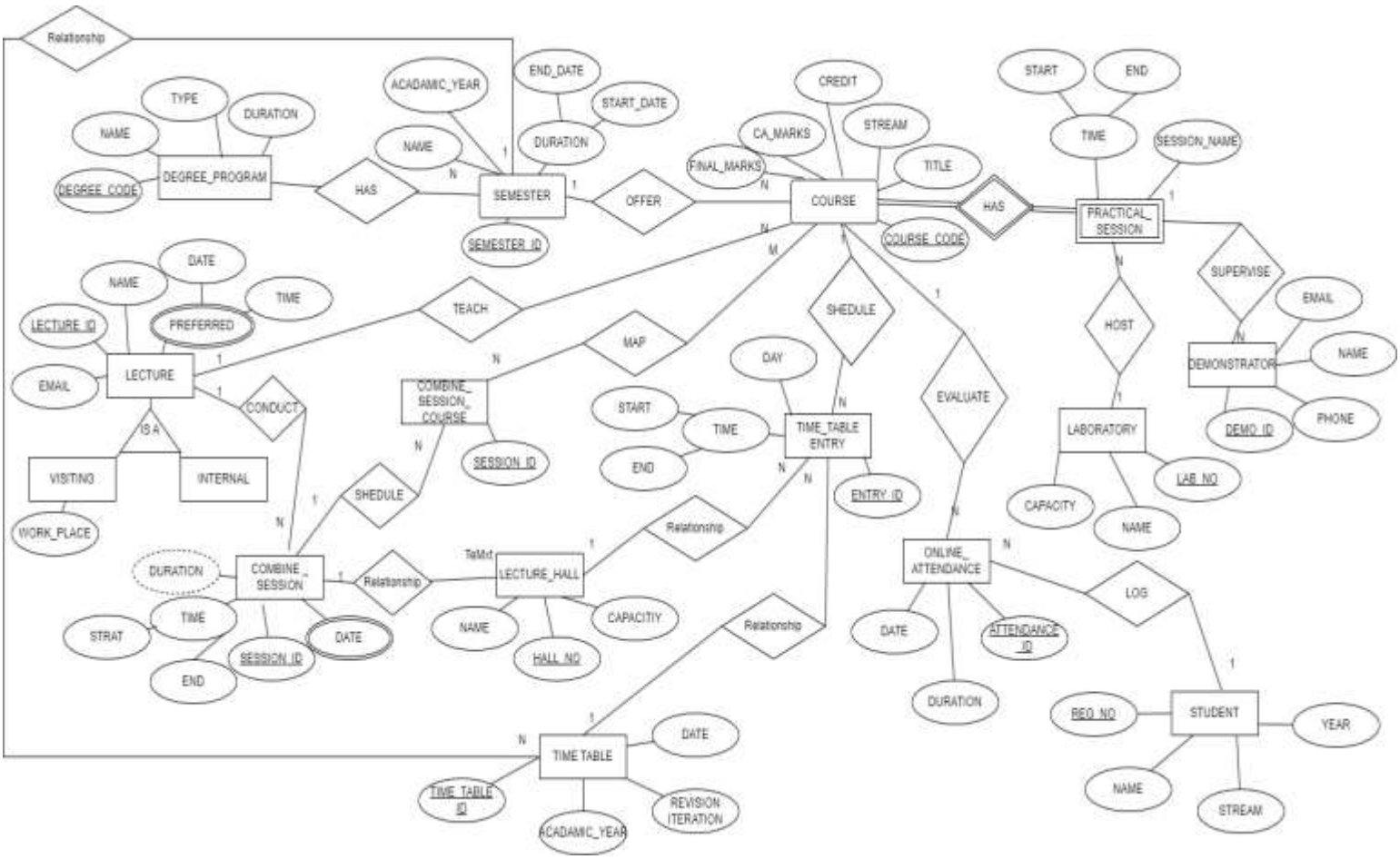
Item	Details
Chosen scenario	Semester-Management-System-Database
Technology stack	MySQL
Source code	https://github.com/akindudulwan/Semester-Management-System-Database

1.0 CONCEPTUAL DATABASE DESIGN

Explains the main entities, attributes, and relationships required for the Semester Management System.

1.1 Entity Audit and Attribute Specification

Identifies the entities, their primary keys, and the attributes used to store academic scheduling information.



Entity / Table	Type	Primary Key	Main Attributes
degree_program	Strong	degree_code	degree_name; degree_type; duration_years
semester	Strong	semester_id	degree_code (FK); academic_year; semester_name; start_date; end_date
lecturer	Supertype	lecturer_id	full_name; preferred_day; preferred_time; is_pref_approved_by_hod; lecturer_type; is_active
internal_lecturer	ISA subtype	lecturer_id (PK, FK)	designation; office_room
visiting_lecturer	ISA subtype	lecturer_id (PK, FK)	email (UQ); phone; home_address; main_workplace; can_conduct_online
course	Strong	course_code	course_title; degree_stream; credits; ca_marks; final_marks; semester_id; lecturer_id
lecture_hall	Strong	hall_no	hall_name; capacity
laboratory	Strong	lab_no	lab_name; capacity; designated_stream
demonstrator	Strong	demo_id	full_name; email (UQ); phone
practical_session	Weak	(course_code, session_no)	session_title; lab_no (FK); demo_id (FK); session_day; start_time; end_time
combined_session	Strong / associative	combined_session_id	session_name; hall_no (FK); lecturer_id (FK); day_of_week; start_time; end_time; duration_hours; is_online
combined_session_course	Junction	(combined_session_id, course_code)	both columns are FKs
timetable	Strong	timetable_id	semester_id (FK); academic_year; revision_iteration; status; published_date

timetable_entry	Strong	entry_id	timetable_id (FK); course_code (FK); hall_no (FK); session_day; start_time; end_time; is_online
student	Strong	student_reg_no	full_name; degree_stream; year
online_attendance	Strong	attendance_id	student_reg_no (FK); course_code (FK); session_date; duration_minutes; is_attended

1.2 Enhanced EER Concepts (Weak Entities, ISA Hierarchy and Combined Sessions)

Explains the weak entity, lecturer ISA specialization, and shared teaching sessions represented in the EER diagram.

- **Weak Entity Concept :** practical_session cannot exist without its identifying parentcourse. It has no independent global key; its partial discriminator is session_no. In the EER diagram, it isrepresented by a **double rectangle**, and the identifying relationship has_practical is drawn with a **doublediamond** with total participation (double line).

- **ISA Specialization Hierarchy :** lecturer is specialized into internal_lecturer andvisiting_lecturer with a **Disjoint and Total (d, t)** constraint: each lecturer belongs to exactly one subclass.

- **Combined Sessions (IT and CS Merge):** Visiting lecturers combine IT and CS student cohorts into one sharedlecture. While students are enrolled under distinct course codes (e.g., ict1302 and csc1302), they attend thesame physical or online session in a lecture hall. This is modeled using combined_session connected to coursevia the junction relation combined_session_course.

2.0 LOGICAL DATABASE DESIGN (RELATIONAL SCHEMA MAPPING)

Explains how the EER model is converted into relational tables with primary and foreign keys.

2.1 Formal Mapping Rules Applied

- **Rule 1 — Regular Strong Entities:** Each strong entity is transformed into a relation. All simpleattributes become columns. The entity's primary key becomes the relation's primary key.

- **Rule 2 — Weak Entities :** Each weak entity becomes a relation. The primary key of the owner entity(course_code) is included as a foreign key and combined with the weak entity's partial key (session_no) toform the **Composite Primary Key: (course_code, session_no)**.

- **Rule 3 — Supertype/Subtype ISA Hierarchy — Option 1 :** - Create a relation for thesuperclass lecturer with primary key lecturer_id and all common attributes.- Create relations for each subclass (internal_lecturer, visiting_lecturer) with primary key lecturer_id,which also acts as a Foreign Key referencing lecturer(lecturer_id) with ON DELETE CASCADE.- *Advantage:* Completely eliminates NULL values while supporting subclass-specific attributes and relationships.

- **Rule 4 — Binary 1:N Relationships :** The primary key of the entity on the "1" side is embedded as aforeign key in the relation on the "N" side: - degree_program (1) to semester (N) → degree_code in semester.- semester (1) to course (N) → semester_id in course.- lecturer (1) to course (N) → lecturer_id in course.- timetable (1) to timetable_entry (N) → timetable_id in timetable_entry.

- **Rule 5 — Binary M:N Relationships :** The M:N relationship between combined_session andcourse is mapped into a junction relation combined_session_course with composite primary key(combined_session_id, course_code).

2.2 Final Relational Schemas (with PKs and FKs)

Notation: PK = Primary Key, FK = Foreign Key, UQ = Unique constraint.

1. **degree_program** (**degree_code PK**, degree_name, degree_type, duration_years)

2. **semester** (**semester_id PK**, degree_code **FK** → degree_program.degree_code, academic_year, semester_name, start_date, end_date)
3. **lecturer** (**lecturer_id PK**, full_name, preferred_day, preferred_time, is_pref_approved_by_hod, lecturer_type, is_active)

4. **internal_lecturer** (**lecturer_id PK, FK** → lecturer.lecturer_id, designation, office_room)

5. **visiting_lecturer** (**lecturer_id PK, FK** → lecturer.lecturer_id, email **UQ**, phone, home_address, main_workplace, can_conduct_online)

6. **course** (**course_code PK**, course_title, degree_stream, credits, ca_marks, final_marks, semester_id **FK** → semester.semester_id, lecturer_id **FK** → lecturer.lecturer_id)

7. **lecture_hall** (**hall_no PK**, hall_name, capacity)

8. **laboratory** (**lab_no PK**, lab_name, capacity, designated_stream)

9. **demonstrator** (**demo_id PK**, full_name, email, phone)

10. **practical_session** (**course_code PK, FK** → course.course_code, session_no **PK**, session_title, lab_no **FK** → laboratory.lab_no, demo_id **FK** → demonstrator.demo_id, session_day, start_time, end_time)

11. **combined_session** (**combined_session_id PK**, session_name, hall_no **FK** → lecture_hall.hall_no, lecturer_id **FK** → lecturer.lecturer_id, day_of_week, start_time, end_time, duration_hours, is_online)

12. **combined_session_course** (**combined_session_id PK, FK** → combined_session.combined_session_id, course_code **PK, FK** → course.course_code)

13. **timetable** (**timetable_id PK**, semester_id **FK** → semester.semester_id, academic_year, revision_iteration, status, published_date)

14. **timetable_entry** (**entry_id PK**, timetable_id **FK** → timetable.timetable_id, course_code **FK** → course.course_code, hall_no **FK** → lecture_hall.hall_no, session_day, start_time, end_time, is_online)

15. **student** (**student_reg_no PK**, full_name, degree_stream, year)

16. **online_attendance** (**attendance_id PK**, student_reg_no **FK** → student.student_reg_no, course_code **FK** → course.course_code, session_date, duration_minutes, is_attended)

3.0 SCHEMA REFINEMENT and NORMALIZATION

Explains how the tables are organized to reduce duplicate data and maintain consistency.

3.1 Functional Dependencies (FDs) Analysis

Table	Primary Key	Other Columns	Foreign-Key References

degree_program	degree_code	degree_name, degree_type, duration_years	—
semester	semester_id	degree_code, academic_year, semester_name, start_date, end_date	degree_code → degree_program
lecturer	lecturer_id	full_name, preferred_day, preferred_time, is_pref_approved_by_hod, lecturer_type, is_active	—
internal_lecturer	lecturer_id	designation, office_room	lecturer_id → lecturer
visiting_lecturer	lecturer_id	email (UQ), phone, home_address, main_workplace, can_conduct_online	lecturer_id → lecturer
course	course_code	course_title, degree_stream, credits, ca_marks, final_marks, semester_id, lecturer_id	semester_id → semester; lecturer_id → lecturer
lecture_hall	hall_no	hall_name, capacity	—
laboratory	lab_no	lab_name, capacity, designated_stream	—
demonstrator	demo_id	full_name, email (UQ), phone	—
practical_session	course_code + session_no	session_title, lab_no, demo_id, session_day, start_time, end_time	course_code → course; lab_no → laboratory; demo_id → demonstrator
combined_session	combined_session_id	session_name, hall_no, lecturer_id, day_of_week, start_time, end_time, duration_hours, is_online	hall_no → lecture_hall; lecturer_id → lecturer
combined_session_course	combined_session_id + course_code	—	combined_session_id → combined_session;

			course_code → course
timetable	timetable_id	semester_id, academic_year, revision_iteration, status, published_date	semester_id → semester
timetable_entry	entry_id	timetable_id, course_code, hall_no, session_day, start_time, end_time, is_online	timetable_id → timetable; course_code → course; hall_no → lecture_hall
student	student_reg_no	full_name, degree_stream, year	—
online_attendance	attendance_id	student_reg_no, course_code, session_date, duration_minutes, is_attended	student_reg_no → student; course_code → course

3.2 Step-by-Step Normalization Proofs

- **First Normal Form (1NF) Proof:** A relation is in 1NF if every field contains one value of the intended type withno repeating groups. In our schema, multi-valued lecturer contacts, multiple courses in combinedlectures, and multiple timetable slots have been fully factored into separate child tables. All domain values areatomic. ⇒ **1NF Holds**.

- **Second Normal Form (2NF) Proof:**
  A relation is in 3NF if it is in 2NF and no non-prime attribute is transitivelydependent on a candidate key through a non-key attribute ($X \to Y \to Z$,). - Hall capacities arestored only in lecture_hall (not duplicated in timetable entry). - Lab capacities are stored only in laboratory (not repeated in practical_session). - Course credits and marks live only in course

- **Third Normal Form (3NF) Proof:** A relation is in 3NF if it is in 2NF and no non-prime attribute is transitivelydependent on a candidate key through a non-key attribute .- Hall capacities arestored only in lecture_hall (not duplicated in timetable_entry).- Lab capacities are stored only in laboratory (not repeated in practical_session).- Course credits and marks live only in course.⇒ **3NF Holds**.

- **Boyce-Codd Normal Form (BCNF) Proof:** A relation is in BCNF if for every non-trivial functional dependency $X\to Y$, the determinant X is a superkey . In all

relations defined above, every determinant on theleft-hand side is a candidate key / superkey. Decompositions are guaranteed to be **Lossless Join** and**Dependency Preserving**. ⇒ **BCNF Holds**.

4.0 PHYSICAL DATABASE DESIGN and IMPLEMENTATION

Explains how the database is implemented in MySQL, including storage and indexing decisions.

4.1 Physical Storage Decisions

- **Storage Engine:** ENGINE = InnoDB. Selected for ACID transaction safety, foreign key referential integritycascading, row-level locking for multi-user timetable drafting, and automatic primary key clustered B+ treeorganization .

- **B+ Tree Indexing Strategy :** - **Clustered Index:** Primary keys automatically form a balancedB+ tree on disk.- **Composite Unique Index (Hall Clash Prevention):** create unique index idx_hall_clash_preventionon timetable_entry (hall_no, session_day, start_time, end_time);Enables the DBMS optimizer to verify hall availability in $O(\log N)$ time, preventing double bookings across L8,L11, and Auditorium Hall.- **Secondary Practical Index:** create index idx_practical_wednesday on practical_session (lab_no,session_day, start_time);

4.2 Complete MySQL 8.0 DDL and DML Implementation

```sql
-- Rajarata University of Sri Lanka
-- Faculty of Applied Sciences
-- Department of Computing
-- Group Project - Binary Brains
-- Semester Management Database

-- create database
create database rjt_as_sm_db;
use rjt_as_sm_db;

-- degree program
create table degree_program (
    degree_code varchar(20) primary key,
    degree_name varchar(120) not null,
    degree_type enum('general', 'honours') not null default 'honours',
    duration_years enum ('3','4') not null
) engine innodb;

-- semester
create table semester (
    semester_id varchar(20) primary key,
    degree_code varchar(20) not null,
    academic_year varchar(15) not null,
    semester_name varchar(50) not null,
    start_date date not null,
    end_date date not null,
    foreign key (degree_code) references degree_program(degree_code) on update cascade on delete restrict,
    check (end_date > start_date),
    unique (degree_code, academic_year, semester_name)
) engine innodb;

-- lecturer superclass and subclasses
create table lecturer (
    lecturer_id varchar(20) primary key,
    full_name varchar(150) not null,
    preferred_day enum ('monday', 'tuesday', 'thursday', 'friday') null,
    preferred_time varchar(50) null,
    is_pref_approved_by_hod boolean not null default false,
    lecturer_type enum ('internal', 'visiting') not null,
    is_active boolean not null default true
) engine innodb;

```

```sql
create table internal_lecturer (
    lecturer_id varchar(20) primary key,
    designation varchar(80) not null,
    office_room varchar(30) null,
    foreign key (lecturer_id) references lecturer (lecturer_id) on update cascade on delete cascade
) engine innodb;

create table visiting_lecturer (
    lecturer_id varchar(20) primary key,
    email varchar(50) not null unique,
    phone varchar(15) not null,
    home_address text null,
    main_workplace varchar(150) null,
    can_conduct_online boolean not null default true,
    foreign key (lecturer_id) references lecturer (lecturer_id) on update cascade on delete cascade
) engine innodb;


--
-- course
create table course (
    course_code varchar(20) primary key,
    course_title varchar(150) not null,
    degree_stream enum('it', 'cs') not null,
    credits int not null,
    ca_marks int not null default 40 check (ca_marks in (30, 40)),
    final_marks int not null default 60 check (final_marks in (60, 70)),
    semester_id varchar(20) not null,
    lecturer_id varchar(20) not null,
    foreign key (semester_id) references semester(semester_id) on update cascade on delete restrict,
    foreign key (lecturer_id) references lecturer (lecturer_id) on update cascade on delete restrict,
    check ((ca_marks + final_marks) = 100)
) engine innodb;

-- lecture halls
create table lecture_hall (
    hall_no varchar(20) primary key,
    hall_name varchar(100) not null,
    capacity int not null check (capacity > 0),
    check (hall_no in ('L8', 'L11', 'auditorium hall'))
) engine innodb;

```

```
82
83      -- laboratories
84  ●   create table laboratory (
85          lab_no varchar(20) primary key,
86          lab_name varchar(100) not null,
87          capacity int not null check (capacity > 0),
88          designated_stream enum('it', 'cs') not null
89      ) engine innodb;
90
91      -- demonstrator
92  ●   create table demonstrator (
93          demo_id varchar(20) primary key,
94          full_name varchar(150) not null,
95          email varchar(120) not null unique,
96          phone varchar(20) not null
97      ) engine innodb;
98
```

```
99      -- practical session
100 ●   create table practical_session (
101         course_code varchar(20) not null,
102         session_no varchar(20) not null,
103         session_title varchar(120) not null,
104         lab_no varchar(20) not null,
105         demo_id varchar(20) not null,
106         session_day enum('monday', 'tuesday','wednesday','thursday', 'friday') not null ,
107         start_time time not null,
108         end_time time not null,
109         primary key (course_code, session_no),
110         foreign key (course_code) references course (course_code) on update cascade on delete cascade,
111         foreign key (lab_no) references laboratory(lab_no) on update cascade on delete restrict,
112         foreign key (demo_id) references demonstrator(demo_id) on update cascade on delete restrict,
113         check (end_time > start_time)
114     ) engine innodb;
115
```

```sql
-- combined session
create table combined_session (
    combined_session_id varchar(20) primary key,
    session_name varchar(150) not null,
    hall_no varchar(20) not null,
    lecturer_id varchar(20) not null,
    day_of_week enum('monday', 'tuesday', 'thursday', 'friday') not null,
    start_time time not null,
    end_time time not null,
    duration_hours decimal (3,1) not null default 2.0,
    is_online boolean not null default true,
    foreign key (hall_no) references lecture_hall(hall_no) on update cascade on delete restrict,
    foreign key (lecturer_id) references lecturer (lecturer_id) on update cascade on delete restrict,
    check (end_time > start_time)
) engine innodb;

create table combined_session_course (
    combined_session_id varchar(20) not null,
    course_code varchar(20) not null,
    primary key (combined_session_id, course_code),
    foreign key (combined_session_id) references combined_session(combined_session_id) on update cascade on delete cascade,
    foreign key (course_code) references course (course_code) on update cascade on delete cascade
) engine innodb;


-- timetable and timetable_entry
create table timetable (
    timetable_id varchar(20) primary key,
    semester_id varchar(20) not null,
    academic_year varchar(15) not null,
    revision_iteration int not null default 1 check (revision_iteration between 1 and 10),
    status enum('draft', 'in_revision', 'approved', 'published') not null default 'draft',
    published_date date null,
    foreign key (semester_id) references semester (semester_id) on update cascade on delete restrict
) engine innodb;


create table timetable_entry (
    entry_id varchar(20) primary key,
    timetable_id varchar(20) not null,
    course_code varchar(20) not null,
    hall_no varchar(20) not null,
    session_day enum('monday', 'tuesday', 'wednesday', 'thursday', 'friday') not null,
    start_time time not null,
    end_time time not null,
    is_online boolean not null default false,
    foreign key (timetable_id) references timetable(timetable_id) on update cascade on delete cascade,
    foreign key (course_code) references course (course_code) on update cascade on delete cascade,
    foreign key (hall_no) references lecture_hall (hall_no) on update cascade on delete restrict,
    check (end_time > start_time),
    check (session_day <> 'wednesday')
) engine innodb;
```

```
166
167     -- student and online attendance
168 •   create table student (
169         student_reg_no varchar(25) primary key,
170         full_name varchar(150) not null,
171         degree_stream enum('it', 'cs') not null,
172         year varchar (10) not null
173     ) engine innodb;
174
175 •   create table online_attendance (
176         attendance_id varchar(20) primary key,
177         student_reg_no varchar(25) not null,
178         course_code varchar(20) not null,
179         session_date date not null,
180         duration_minutes int not null check (duration_minutes >= 0),
181         is_attended boolean generated always as (duration_minutes >= 40) stored,
182         foreign key (student_reg_no) references student (student_reg_no) on update cascade on delete cascade,
183         foreign key (course_code) references course(course_code) on update cascade on delete cascade
184     ) engine innodb;
185

185
186     -- indexes
187 •   create unique index idx_hall_clash_prevention on timetable_entry (hall_no, session_day, start_time, end_time);
188 •   create index idx_practical_wednesday on practical_session (lab_no, session_day, start_time);
189
```

5.0 DATABASE SECURITY DESIGN (RBAC)

Explains how user roles and permissions protect database information.

Role	Degree Programme, Semester, Course, Halls and Labs	Timetable and Timetable Entry	Student and Attendance
Dean	SELECT (Read-only)	SELECT (Read-only)	SELECT (Read-only)
AR Officer	SELECT (Read-only)	SELECT, INSERT, UPDATE (Read only and Data entry)	SELECT (Read-only)
HOD	SELECT, INSERT, UPDATE, DELETE (Full access)	SELECT, INSERT, UPDATE, DELETE (Approval and management)	SELECT, INSERT, UPDATE, DELETE (Full access)
Clerk	SELECT, INSERT, UPDATE, DELETE (Data entry)	SELECT (View only)	SELECT, INSERT, UPDATE, DELETE (Data entry)

5.2 Security Implementation SQL Script (Roles and Dedicated Users)

```
190
191      -- Security of database
192      --  Create Roles
193 •    create role if not exists role_dean;
194 •    create role if not exists role_ar_officer;
195 •    create role if not exists role_hod;
196 •    create role if not exists role_clerk;
197
198      --  Grant Permissions to Roles
199 •    grant select on rjt_as_sm_db.* to role_dean;
200
201 •    grant select on rjt_as_sm_db.* to role_ar_officer;
202 •    grant insert, update on rjt_as_sm_db.timetable to role_ar_officer;
203 •    grant insert, update on rjt_as_sm_db.timetable_entry to role_ar_officer;
204
205 •    grant select, insert, update, delete on rjt_as_sm_db.* to role_hod;
206
207 •    grant select, insert, update, delete on rjt_as_sm_db.student to role_clerk;
208 •    grant select, insert, update, delete on rjt_as_sm_db.lecturer to role_clerk;
209 •    grant select, insert, update, delete on rjt_as_sm_db.course to role_clerk;
210

210
211      -- Create Users with Passwords
212 •    create user if not exists 'dean_fas'@'localhost' identified by 'dean123!';
213 •    create user if not exists 'hod_comp'@'localhost' identified by 'hod123!';
214 •    create user if not exists 'ar_office'@'localhost' identified by 'ar123!';
215 •    create user if not exists 'clerk_comp'@'localhost' identified by 'clerk123!';
216
217      -- Assign Roles to Users
218 •    grant role_dean to 'dean_fas'@'localhost';
219 •    grant role_hod to 'hod_comp'@'localhost';
220 •    grant role_ar_officer to 'ar_office'@'localhost';
221 •    grant role_clerk to 'clerk_comp'@'localhost';
222

222
223      -- Activate Default Roles for Users
224 •    set default role all to 'dean_fas'@'localhost';
225 •    set default role all to 'hod_comp'@'localhost';
226 •    set default role all to 'ar_office'@'localhost';
227 •    set default role all to 'clerk_comp'@'localhost';
228
229 •    flush privileges;
230
```

Conclusion

The Semester Management System database design provides a structured approach to managing academic semester information, courses, lecturer preferences, timetable preparation, practical sessions, and venue allocation. The EER model identifies the main entities, their attributes, and the relationships required to support the system's operations. The relational schema translates this model into database tables using primary keys and foreign keys to maintain data integrity and consistency.

Database normalization helps reduce unnecessary data duplication, while the implementation of Role-Based Access Control (RBAC) supports secure access according to the responsibilities of each user. The proposed database design provides a foundation for organizing semester scheduling information, managing timetable revisions, and reducing scheduling conflicts. Overall, the Semester Management System database aims to improve the accuracy, consistency, and efficiency of semester and timetable management at the Department of Computing, Rajarata University of Sri Lanka.